

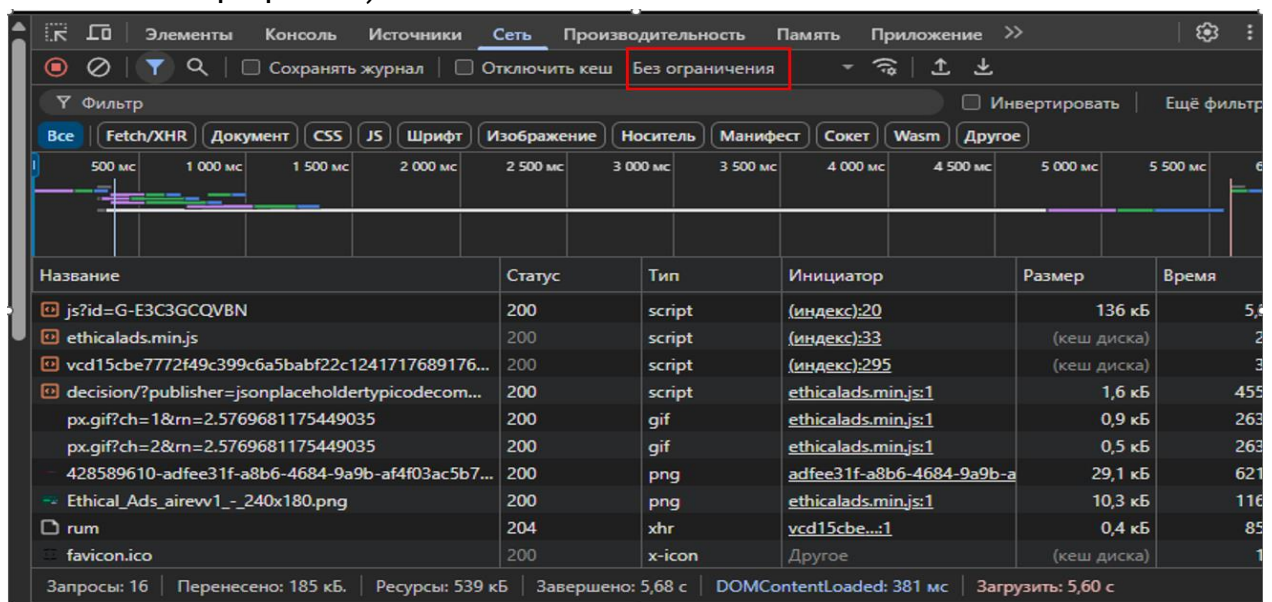
Практическая работа №1

Пользовательские запросы в веб-приложении (DevTools, HTTP, REST, JSON, Postman)

2. Задание

2.1. Требуемые инструменты

- браузер (Chrome / Firefox / Edge / Яндекс.Браузер);
- DevTools → вкладка **Network** (режим Online, без ограничения скорости или с 4G-профилем).



2.2. Первичный анализ запросов

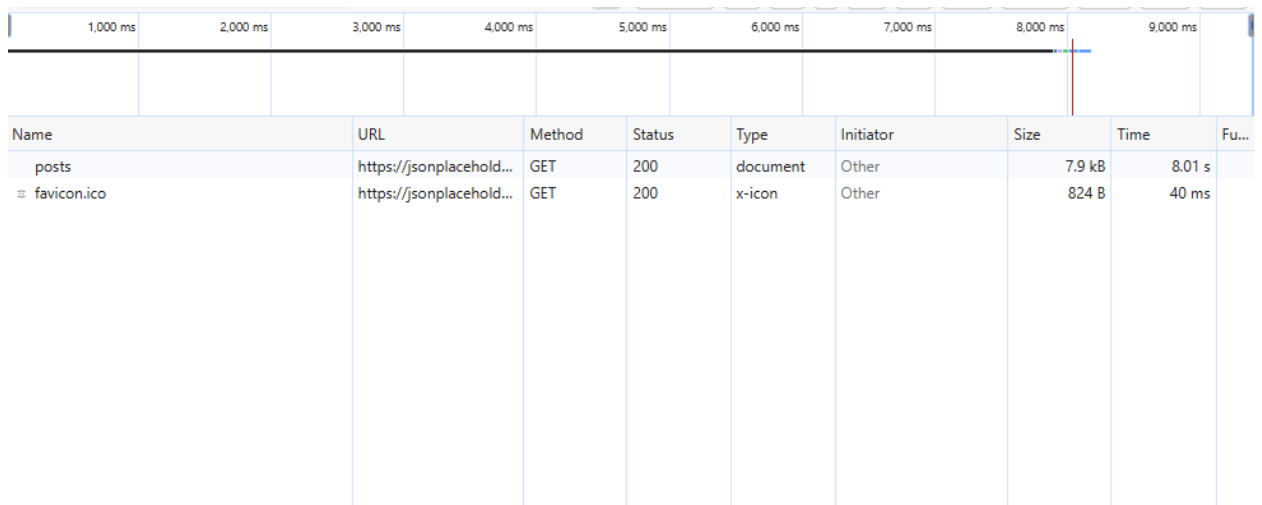
1. Откройте DevTools (**F12**) → вкладка **Network**.
2. Загрузите страницу:

`https://jsonplaceholder.typicode.com/posts`

1. Зафиксируйте в таблице: метод, URL, код ответа, размер данных, время загрузки.

Имя запроса	Метод	URL	Код ответа	Размер данных	Время загрузки
posts	GET	https://jsonplaceholder.typicode.com/posts	200	7.9 kb	8.01 s
Favicon.ico	GET	https://jsonplaceholder.typicode.com/favicon.ico	200	824B	40ms

Сделайте скриншот списка запросов.



Name	URL	Method	Status	Type	Initiator	Size	Time	Fu...
posts	https://jsonplaceholder...	GET	200	document	Other	7.9 kB	8.01 s	
≡ favicon.ico	https://jsonplaceholder...	GET	200	x-icon	Other	824 B	40 ms	

2.3. Детальный разбор одного запроса

Выберите любой запрос и опишите:

- метод;
- URL и параметры;
- заголовки (Request Headers / Response Headers);
- тело ответа (Response / Preview);
- код ответа;
- время выполнения.

Сравните его с другим запросом и сделайте вывод.

- 1) Метод: GET
- 2) URL <https://jsonplaceholder.typicode.com/posts>
- 3) Request headers:

authority, method, path. Scheme, accept

▼ Request headers	
:authority	jsonplaceholder.typicode.com
:method	GET
:path	/posts
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru,en;q=0.9,en-GB;q=0.8,en-US;q=0.7,ru-RU;q=0.6
Cache-Control	max-age=0
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="154", "Microsoft Edge";v="154", "Not A(Brand";v="99"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0

Response headers: Cache-control, Content-type. X-Powered-Limit

▼ response headers	
Access-Control-Allow-Credentials	true
Age	27776
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a45ea5904ad4b79a-ARN
Content-Encoding	gzip
Content-Type	application/json; charset=utf-8
Date	Mon, 05 Oct 2026 18:56:30 GMT
Etag	W/"6b80-Ybsq/K6GwwqrYkAsFxxQXGC7DoM"
Expires	-1
Nel	{"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
Pragma	no-cache
Report-To	{"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a.nel.cloudflare.com/report/v4?s=yF9Pb15wdlTE%2BHoyZc6AGy4fIjsFO0JVDprn1PV534c3YurG93HsjMNO%2BKfp9n5O5RQxIRjyE%2BeYaQIyHuU20sgvr%2BUQU0K6hySuhs0%2FC%2F2B4H4pABhj06dWthAE7s8nthDaJ50L11075r8FLiOI"}]}
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=6,cfOrigin;dur=0
Vary	Origin, Accept-Encoding
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	3600000
X-Ratelimit-Remaining	3599997
X-Ratelimit-Reset	1790812826

4) Тело ответа:
- Preview

```
✕ Headers Preview Response Initiator Timing
Preview
[,...]
▶ 0: {userId: 1, id: 1, title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",...}
▶ 1: {userId: 1, id: 2, title: "qui est esse",...}
▶ 2: {userId: 1, id: 3, title: "ea molestias quasi exercitationem repellat qui ipsa sit aut",...}
▶ 3: {userId: 1, id: 4, title: "eum et est occaecati",...}
▶ 4: {userId: 1, id: 5, title: "nesciunt quas odio",...}
▶ 5: {userId: 1, id: 6, title: "dolorem eum magni eos aperiam quia",...}
▶ 6: {userId: 1, id: 7, title: "magnam facilis autem",...}
▶ 7: {userId: 1, id: 8, title: "dolorem dolore est ipsam",...}
▶ 8: {userId: 1, id: 9, title: "nesciunt iure omnis dolorem tempora et accusantium",...}
▶ 9: {userId: 1, id: 10, title: "optio molestias id quia eum",...}
▶ 10: {userId: 2, id: 11, title: "et ea vero quia laudantium autem",...}
▶ 11: {userId: 2, id: 12, title: "in quibusdam tempore odit est dolorem",...}
▶ 12: {userId: 2, id: 13, title: "dolorum ut in voluptas mollitia et saepe quo animi",...}
▶ 13: {userId: 2, id: 14, title: "voluptatem eligendi optio",...}
▶ 14: {userId: 2, id: 15, title: "eveniet quod temporibus",...}
▶ 15: {userId: 2, id: 16, title: "sint suscipit perspiciatis velit dolorum rerum ipsa laboriosam odio",...}
▶ 16: {userId: 2, id: 17, title: "fugit voluptas sed molestias voluptatem provident",...}
▶ 17: {userId: 2, id: 18, title: "voluptate et itaque vero tempora molestiae",...}
▶ 18: {userId: 2, id: 19, title: "adipisci placeat illum aut reiciendis qui",...}
▶ 19: {userId: 2, id: 20, title: "doloribus ad provident suscipit at",...}
▶ 20: {userId: 3, id: 21, title: "asperiores ea ipsam voluptatibus modi minima quia sint",...}
▶ 21: {userId: 3, id: 22, title: "dolor sint quo a velit explicabo quia nam",...}
▶ 22: {userId: 3, id: 23, title: "maxime id vitae nihil numquam",...}
▶ 23: {userId: 3, id: 24, title: "autem hic labore sunt dolores incidunt",...}
▶ 24: {userId: 3, id: 25, title: "rem alias distinctio quo quis",...}
▶ 25: {userId: 3, id: 26, title: "est et quae odit qui non",...}
▶ 26: {userId: 3, id: 27, title: "quasi id et eos tenetur aut quo autem",...}
▶ 27: {userId: 3, id: 28, title: "delectus ullam et corporis nulla voluptas sequi",...}
▶ 28: {userId: 3, id: 29, title: "iusto eius quod necessitatibus culpa ea",...}
▶ 29: {userId: 3, id: 30, title: "a quo magni similique perferendis",...}
▶ 30: {userId: 4, id: 31, title: "ullam ut quidem id aut vel consequuntur",...}
▶ 31: {userId: 4, id: 32, title: "doloremque illum aliquid sunt",...}
▶ 32: {userId: 4, id: 33, title: "qui explicabo molestiae dolorem",...}
▶ 33: {userId: 4, id: 34, title: "magnam ut rerum iure",...}
▶ 34: {userId: 4, id: 35, title: "id nihil consequatur molestias animi provident",...}
```

- Response:

```
✕ Headers Preview Response Initiator Timing
Response
1 [
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet",
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reicien",
13  },
14  {
15    "userId": 1,
16    "id": 3,
17    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et lab",
19  },
20  {
21    "userId": 1,
22    "id": 4,
23    "title": "eum et est occaecati",
24    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam i",
25  },
26  {
27    "userId": 1,
28    "id": 5,
29    "title": "nesciunt quas odio",
30    "body": "repudiandae veniam quaerat sunt sed\nalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nest aut tenetur dolor neque",
31  },
32  ]
33  ]
```

5) Код ответа: 200

6) Время выполнения: 8.01 s

Вывод: /posts передаёт 7.9 kb данных и грузится 8 секунд. /favicon.ico 824 байта и 40 миллисекунд. Чем больше данных и чем сложнее их получить, тем дольше запрос.

2.4. Эксперименты с запросами

Выполните в консоли:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(res => res.json())
  .then(data => console.log(data));
```

```
> fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(res => res.json())
  .then(data => console.log(data));
< ▶ Promise {<pending>}

▶ {userId: 1, id: 1, title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', body:
  'quia et suscipit\nsuscipit recusandae consequuntur urna strum rerum est autem sunt rem eveniet architecto'}
```

Проанализируйте запрос во вкладке Network. Сравните:

Request #	Name	URL	Method	Status	Type	Initiator	Size	Time
3	1	https://jsonplaceholder.typ...	GET	200	fetch	VM13:1	989 B	204 ms

- GET /posts (все посты);
- GET /posts/1 (один пост).

Сделайте выводы по:

- размеру ответа;
- времени загрузки;
- количеству записей.

	Размер ответа	Время загрузки	Кол-во записей
GET /posts	7.9 KB	8.01 сек	100
GET /posts/1	989 B	204 ms	1

1. Вывод:

Размер ответа: GET /posts 7.9 KB, GET /posts/1 989 B. Разница в 8 раз. /posts возвращает список всех записей, /posts/1 только одну запись. Размер ответа зависит от количества данных, больше записей больше байт

Время загрузки: GET /posts 8.01 с, GET /posts/1 204 мс. чем больше данных и сложнее запрос, тем дольше сервер обрабатывает запрос и дольше передаётся ответ.

Количество записей: GET /posts 100 записей, GET /posts/1 1 запись.
/posts возвращает все ресурсы, /posts/1 конкретный ресурс по id

2.5. Работа с параметрами

1. Откройте в браузере:

<https://jsonplaceholder.typicode.com/posts?userId=1>

Request #	Name	URL	Method	Status	Type	Initiator	Size	Time	Content-Type	Ful...
3	posts?userId=1	https://jsonplaceholder.typ...	GET	200	document	Other	1.9 kB	136 ms		

1. Сравните с <https://jsonplaceholder.typicode.com/posts>:

2. сколько объектов вернулось;

3. размер ответа;

4. время загрузки.

Сделайте вывод, как параметры помогают уменьшить объём данных.

	Объектов	Размер ответа	Время загрузки
/posts	100	7.9 KB	8.01 с
/posts?userId=1	10	1.9 kb	136 ms

Вывод: Параметры помогают сузить выборку(получить только нужное) соответственно сократить размер ответа что уменьшает объем данных и увеличивает скорость загрузки

2.6. Массовые запросы

Откройте несколько URL одновременно:

- `/posts/1`
- `/posts/2`
- `/posts/3`

```
> fetch("https://jsonplaceholder.typicode.com/posts/1") .then(res => res.json()) .then(data => console.log(data)); fetch("https://jsonplaceholder.typicode.com/posts/2") .then(res => res.json()) .then(data => console.log(data)); fetch("https://jsonplaceholder.typicode.com/posts/3") .then(res => res.json()) .then(data => console.log(data));
```

» Promise {<pending>}

» {userId: 1, id: 1, title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', body: 'quia et suscipit\nsuscipit recusandae consequuntur \u2014strum rerum est autem sunt rem eveniet architecto'} [VM12:1](#)

» {userId: 1, id: 2, title: 'qui est esse', body: 'est rerum tempore vitae\nsequi sint nihil reprehend\aperiam non debitis possimus qui neque nisi nulla'} [VM12:1](#)

» {userId: 1, id: 3, title: 'ea molestias quasi exercitationem repellat qui ipsa sit aut', body: 'et iusto sed quo iure\nvoluptatem occaecati omnis e\nmolestiae porro eius odio et labore et velit aut'} [VM12:1](#)

Зафиксируйте:

- как меняется время отклика;
- как браузер параллелит запросы;
- нет ли избыточных запросов.

Request #	Name	URL	Method	Status	Type	Initiator	Size	Time	Content-Type	Ful...
3	posts?userId=1	https://jsonplaceholder.typ...	GET	200	document	Other	1.9 kB	136 ms		
2	 1	https://jsonplaceholder.typ...	GET	304	fetch	VM12:1	738 B	134 ms		
3	 2	https://jsonplaceholder.typ...	GET	200	fetch	VM12:1	974 B	134 ms		
4	 3	https://jsonplaceholder.typ...	GET	200	fetch	VM12:1	977 B	138 ms		

Время запроса	
<code>/posts/1</code>	134 ms
<code>/posts/2</code>	134 ms
<code>/posts/3</code>	138 ms

Время запросов почти одинаковое, что говорит о том что они отправляются параллельно. Избыточных запросов нет.

2.7. Заголовки и ответы

1. Во вкладке **Headers** выпишите не менее 5 ключевых заголовков, например:
2. `Content-Type`
3. `Cache-Control`
4. `Etag`

5. `Date`
6. `Server`
7. Сравните вкладки **Preview** и **Response**:
8. чем они отличаются по представлению данных;
9. в каких случаях удобнее использовать Preview, а в каких — Response.

1. Server: CloudFlare
2. Date: Mon, 05 Oct 2026 20:43:18 GMT
3. Cache Control: max-age=43200
4. X-Content-type-options: nosniff
5. ETag: W/"124-yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"

Preview - это обработанный браузером ответ. Response - это сырой текст ответа, каким он пришел по сети.

Preview удобен для анализа структуры и быстрого просмотра, а Response для точного сырого ответа, копирования и проверки формата

2.8. Оптимизация запросов

Сравните ответы:

Запрос	Размер ответа	Время загрузки
<code>/posts</code>	7.9 KB	8.01 с
<code>/posts?userId=1</code>	1.9 kb	136 ms
<code>/posts/1</code>	989 B	204 ms

Предложите меры оптимизации:

- пагинация (ограничение количества записей на страницу);
- фильтрация (по `userId`, статусам и т.п.);
- уменьшение размера ответа (убрать лишние поля);

- кэширование на клиенте и сервере;
- использование правильных заголовков (Cache-Control, ETag и т.д.).

2.9. Работа с API через Postman

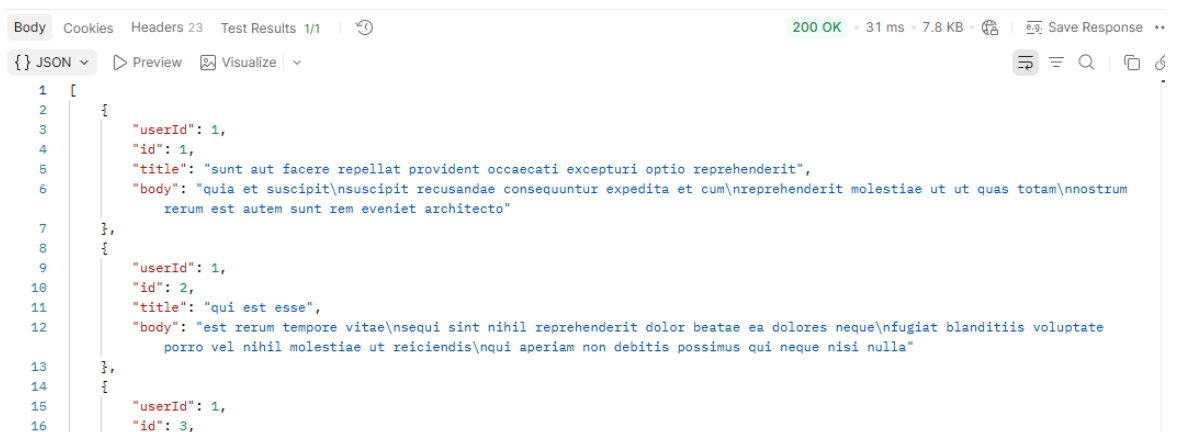
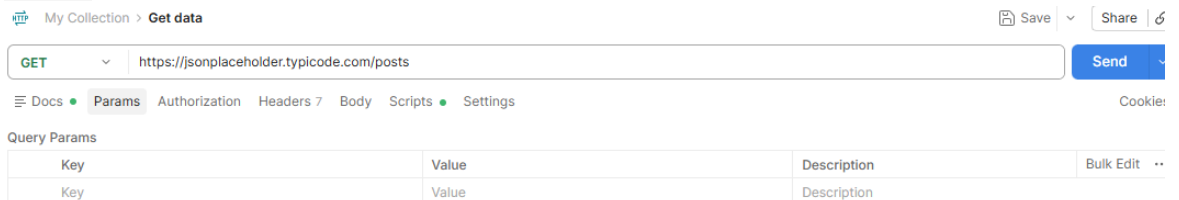
1. Перейдите на сайт:

<https://www.postman.com/>

2. Создайте новый workspace.

3. Выполните:

1. GET /posts



2. GET /posts/1

The screenshot shows the Postman interface for a GET request to `https://jsonplaceholder.typicode.com/posts/1`. The request is successful, returning a 200 OK status with a response time of 28 ms and a body size of 1.12 KB. The response body is a JSON object:

```
{  "userId": 1,  "id": 1,  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"}
```

Сравните ответы.

1. Выполните **POST** /posts с JSON-телом:

```
{  "title": "Пост через Postman",  "body": "Текст поста",  "userId": 2}
```

1. Выполните:

На сервер отправляется запрос с данными и создается пост. Получаем

следующий ответ:

POST ▼ https://jsonplaceholder.typicode.com/posts Send ▼

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 {
2
3   "title": "Пост через Postman",
4
5   "body": "Текст поста",
6
7   "userId": 2
8 }
9
```

Body Cookies Headers 23 Test Results 1/1 🔄 **201 Created** • 36 ms • 1.12 KB • 📄 Save Response ⋮

{} JSON ▼ ▶ Preview 🖼️ Visualize ▼ 🔍 📄 🔗

```
1 {
2   "title": "Пост через Postman",
3   "body": "Текст поста",
4   "userId": 2,
5   "id": 101
6 }
```

2. **PUT** /posts/1 — полное обновление ресурса;

PUT ▼ https://jsonplaceholder.typicode.com/posts/1 Send ▼

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

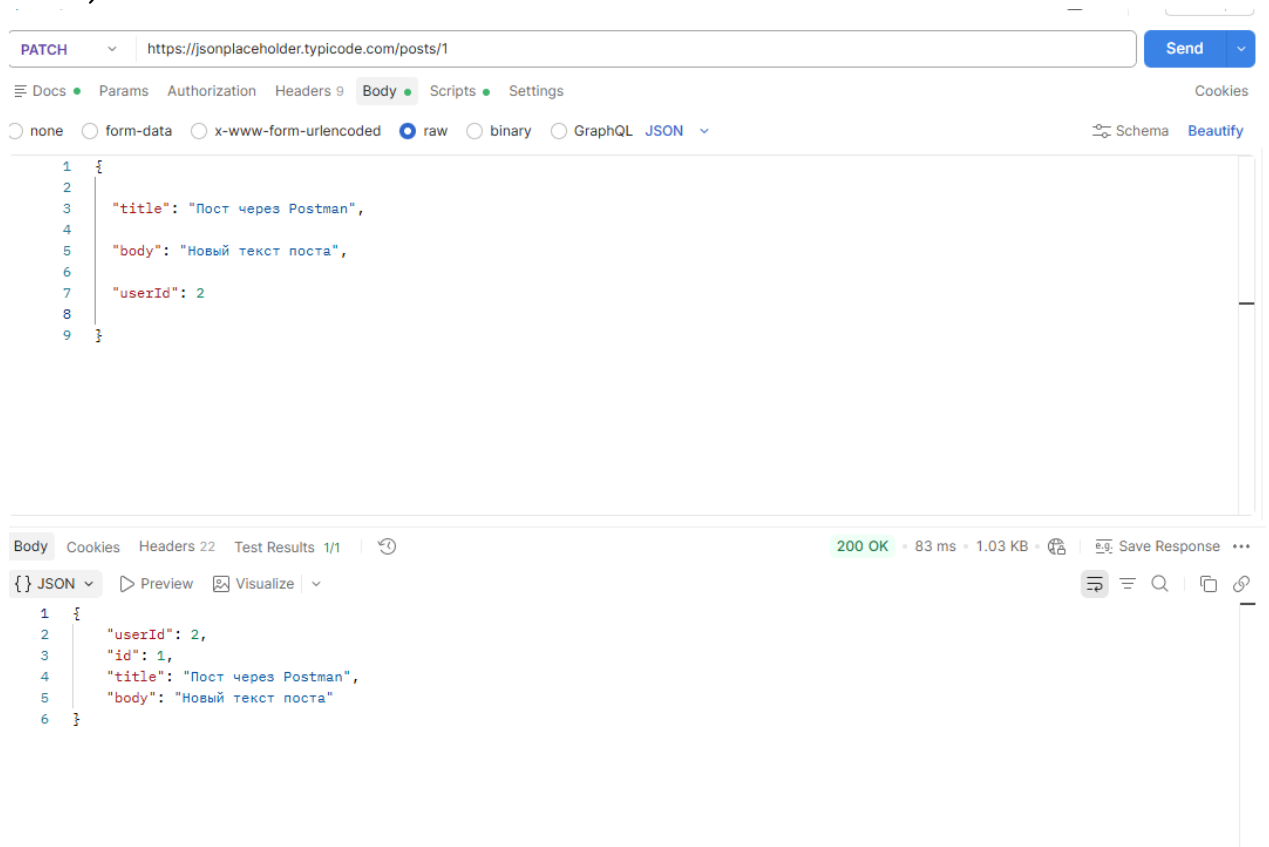
```
1 {
2
3   "title": "Пост через Postman",
4
5   "body": "Текст поста",
6
7   "userId": 2
8 }
9
```

Body Cookies Headers 22 Test Results 1/1 🔄 **200 OK** • 38 ms • 1.01 KB • 📄 Save Response ⋮

{} JSON ▼ ▶ Preview 🖼️ Visualize ▼ 🔍 📄 🔗

```
1 {
2   "title": "Пост через Postman",
3   "body": "Текст поста",
4   "userId": 2,
5   "id": 1
6 }
```

3. **PATCH** /posts/1 — частичное обновление (изменить только один-два поля).



Объясните, чем отличаются результаты.

1. Выполните **DELETE** /posts/1 и зафиксируйте:
2. код ответа
3. тело ответа (если есть).

Код ответа: 200 OK

Тело ответа: Нет

2.10. Сравнение DevTools и Postman

Сделайте таблицу или краткое сравнительное описание:

- **DevTools**
- анализ реальных запросов страницы;
- проверка производительности, кэширования;
- работа в контексте открытого сайта.
- **Postman**
- ручное конструирование запросов;

- сохранение коллекций;
- тесты и скрипты;
- работа с авторизацией и токенами.

Сделайте вывод: **когда удобнее DevTools, а когда Postman.**

Вывод:

DevTools нужен, чтобы посмотреть, как работает готовый сайт, какие запросы отправляются, какие данные приходят, что тормозит страницу, как работает кэширование. Он работает в контексте открытой страницы с её сессией, cookie, авторизацией.

Postman нужен, чтобы создавать и тестировать запросы к API вручную, правильно ли сохраняются данные в POST, правильные ли данные приходят в GET. Он не привязан к сайту, можно отправлять любые запросы к любому API.

3. Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

Ответ:

DevTools это инструменты разработчика в браузере.

Elements HTML/CSS, структура DOM. Console ошибки, логи, выполнение JS. Network, запросы, ответы, время, заголовки, кэш.

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

Ответ: f12 или правой кнопкой мыши вызвать меню и выбрать проверить

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.

Ответ: Метод (GET, POST, PUT, PATCH). URL/параметры - адрес, указание какие данные нужны. Заголовки метаданные (тип, кэш и т.п). Тело данные для POST

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

Ответ:

GET нужен для получения данных,

GET /index.html

POST для отправки данных на сервер,

POST /posts

PUT для полного обновления в отличие от POST он не создает новый ресурс

PUT /posts/1 - полностью обновит пост 1

PATCH для частичного обновления например обновить только одну строку

PATCH /posts/1 - частично обновит указанные строки в посте 1

DELETE для удаления.

DELETE /posts/1 - удалит пост 1

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?

Ответ:

HEAD: возвращает заголовки. Используется, чтобы проверить наличие файла, узнать его размер (Content-Length) или дату последнего изменения.

OPTIONS: Используется, чтобы узнать, какие HTTP-методы поддерживает сервер для конкретного URL (например, разрешен ли DELETE).

Preflight-запрос это служебный запрос OPTIONS, который браузер автоматически отправляет перед "сложными" запросами (не GET/POST с простыми заголовками) при кросс-доменных вызовах (CORS).

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

Ответ:

1xx - информационный

2xx - успех

3xx - перенаправление

4xx - ошибки на стороне клиента

5xx - ошибки на стороне сервера

7. Что такое REST и как ресурс в REST представлен на уровне URL?

Ответ: REST это архитектурный стиль API, где ресурсы представлены URL, а действия HTTP-методами.

Ресурс в URL: /posts, /posts/1, /users/1/posts.

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

Ответ:

JSON типы: строка, число, bool, null, объект, массив.

Пример объекта: {"id":1,"name":"Alex"}.

Пример массива: [{"id":1},{id":2}].

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Ответ:

Preview отформатированное представление (дерево JSON).

Response сырой ответ как от сервера (текст, JSON-строка)

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

Ответ:

Content-Type, Content-Encoding, Cache-Control, ETag, Last-Modified, Authorization, Set-Cookie, CORS-заголовки.

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

Ответ:

Пагинация: Отдавать данные страницами (параметры ?page=2&size=20 или ?offset=100&limit=50).

Фильтрация: запрашивать только нужные данные.

Кэширование (Caching): использовать заголовки Cache-Control и ETag на сервере.

Сжатие (Compression): Включить на сервере сжатие Gzip/Brotli.

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

Ответ:

DevTools анализ реальных запросов открытой страницы, производительность, кэш. Уместен при отладке сайта.

Postman ручное создание и тестирование API, коллекции, токены, автотесты. Уместен при разработке/тестировании API.